

validation and miscellaneous operators. Operators always begin with a @ character, and are always followed by a space.

- **ACTIONS:** Specify what to do if the rule evaluates to 'true'—step on to another rule, display an error message, or any other task. Actions are divided into seven categories: *disruptive, flow, metadata, variable, logging, special and miscellaneous actions*. Here is a simple example of a rule:

```
SecRule ARGS|REQUEST_HEADERS "@rx <script" id:101,msg: 'XSS
Attack', severity:ERROR,deny,status:404
```

Here, *ARGS* and *REQUEST_HEADERS* are variables (request parameters and request headers, respectively); @rx is the operator used to match a pattern in the variables (here, this pattern is <script); id, msg, severity, deny and status are all actions to be performed if the pattern is matched. This rule is used to avoid XSS attacks by checking for a <script pattern in the request parameters and header, and generates an 'XSS Attack' message. The ID 101 is given to the rule; it will deny any matching request with a 404 status response.

Let's look at another example, for more clarity:

```
SecRule ARGS:username "@streq admin" chain,deny
SecRule REMOTE_ADDR "!@streq 192.168.1.1"
```

This is an example of chaining two rules, and the transfer of control to another rule if the first rule holds true. The first rule checks for the string *admin* in the request's *username* parameter. If found, the second rule will be activated, which denies all such requests that are not from the 192.168.1.1 IP address. Thus, chaining rules help to create complex rules.

Now, writing filtering rules for each attack will be very cumbersome, and also prone to human error. Here, *mod_security* provides users with another directive, *SecFilter*. This looks for a keyword in the request. It will be applied to the first line of the request (the one that looks like *GET /index.php?parameter=value HTTP/1.0*). In case of POST requests, the body of the request will be searched too (provided request body buffering is enabled). All pattern matches are case-insensitive, by default. The syntax for *SecFilter* is *SecFilter KEYWORD*.

Rules against major attacks

Let's look at some rules to prevent major attacks on Web applications.

SQL injection

Suppose you have an application that is vulnerable to SQL-injection attacks. An attacker could try to delete all records from a MySQL table, like this:

```
http://www.example.com/login.php?user=arpit';DELETE%20FROM%20
users--
```

This can be prevented with the following directive:

```
SecFilter "delete[:space:]]+from"
```

Whenever such a request is caught by the filter, something similar to the following code is *logged to audit_log*:

```
=====
Request: 192.168.0.207 - - [04/Jul/2006:23:43:00 +1200] "GET /
login.php?user=ton';DELETE%20FROM%20users-- HTTP/1.1" 500 1215
Handler: (null)
-----
GET /login.php?user=arpit';DELETE%20FROM%20users-- HTTP/1.1
Host: 192.168.0.100
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US;
rv:1.8.0.4) Gecko/20060508 Firefox/1.5.0.4
Accept: text/xml,application/xml,application/xhtml+xml,text/
html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
mod_security-message: Access denied with code 500. Pattern match
"delete[:space:]]+from" at THE_REQUEST
mod_security-action: 500

HTTP/1.1 500 Internal Server Error
Last-Modified: Fri, 21 Oct 2005 14:30:18 GMT
ETag: "8238-4bf-833a5280"
Accept-Ranges: bytes
Content-Length: 1215
Connection: close
Content-Type: text/html
```

In response to the attack, *SecFilterDefaultAction* is applied (the request is denied, logged, and the attacker gets a 500 error). If you want a different action to take place (like, redirect the request to a HTML page that can provide customised warning content), you can specify this in the rule, as follows:

```
SecFilter "delete[:space:]]+from" log,redirect:http://example.
com/invalid_request.html
```

To prevent more SQL injection attacks, we can add a few other directives like:

```
SecFilter "insert[:space:]]+into"
SecFilter "select.+from"
SecFilter "drop[:space:]]table"
```